

CPU Simulator 作业发布

RISC-V ■ Tomasulo

陈柯杰

2026.7.27

作业内容

用 C++ 模拟一个 Tomasulo 架构的 RISC-V CPU，通过所有下发数据。

评分标准

测试点 + 理解	75% + 15%
缺陷情况（未实现分支预测 / 主要功能缺陷）	60% + 10%
CR（代码审查）	10%
Bonus	10%

执行流程

- ▶ 从标准输入读入机器指令。
- ▶ 从内存 0x0000 开始取指，每次连取 4 个 byte 拼成一条指令。
- ▶ 执行到 0x0ff00513 (li a0,255) 时，向 stdout 输出 a0 低 8 位 (返回值 0-255)。

读入：用 `map<uint32,uint8>` 或数组模拟内存。遇到 @addr 就跳到该地址，之后逐 byte 写入 `mem[cur++]=byte`。

Final memory (byte view)

0x00000000

37

01

02

00

0x00000004 ... 0x00000FFF (unused, =0)

0x00001000

13

05

F0

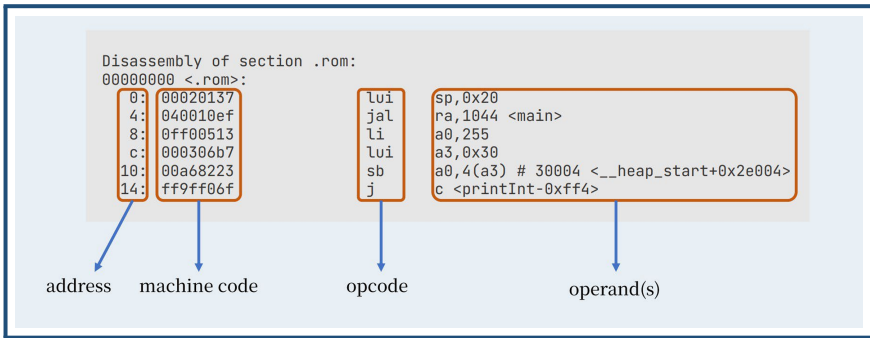
0F

```
@00000000    37 01 02 00
               ; lui sp, 0x20
@00001000    13 05 F0 0F
               ; li a0, 255
```

最终内存：两个 @ 区域分别落在 0x00000000 与 0x00001000，中间地址默认 0。

每个测试程序都包含三个同名文件：

- ▶ .c: 人类可读的 C 源码，用来理解程序在算什么。
- ▶ .data: 你的 simulator 的实际输入，机器码的文本十六进制表示。
- ▶ .dump: 对应的 objdump 反汇编，便于对照每条机器码对应的汇编指令。



示例与最基础的注意事项

(1) 端序示例

在 0x00 (4-byte 对齐) 存 0x01020304,
内存中实际为:

04 03 02 01

(RISC-V 采用小端字节序)

(2) .data 首 4 字节

37 01 02 00 -> 0x00020137 -> lui
sp, 0x20

- ▶ 请务必仔细阅读 RISC-V 手册中的相关内容，理解其特性和规范！
- ▶ 小端字节序：0x01020304 在内存中是 04 03 02 01。
- ▶ 同理，.data 前四个 byte 37 01 02 00 对应指令码 0x00020137。
- ▶ 寄存器 x0 每周期重置回 0。

- ▶ 友情提醒：很多「伪指令」其实是某些指令的特化形式，例如 `li rd,imm = addi rd,x0,imm, j = jal x0,offset`。提前知道这点，读 .dump 反汇编时就不会困惑。
- ▶ 写 decoder 要特别谨慎：务必弄清符号扩展 / 无符号扩展、要截取哪几位等特性；建议给 decoder 写单元测试，逐项验证。

- ▶ 你应当为每个 Tomasulo 架构中的元件写一个类。
- ▶ 这些类中，应至少有一个成员函数，包含这个元件在每个周期中的行为。
- ▶ 尽可能遵循硬件要求，需要统计花费的时钟周期数。
- ▶ 大家写的 C++ 程序是顺序执行的；**但实际硬件中元件是并行执行的。因此要求 module 的执行顺序可以交换。**
- ▶ 实现分支预测，并统计预测准确率。
- ▶ 数据内存读写和指令内存读写可以同时进行。
- ▶ 数据内存访问需模拟硬件的延迟返回，不得直接立即使用全局变量的值。

代码实现参考（示例）

下面给出一份「CPU 运行一步」的参考骨架截图。

```
int main(){
    int clk = 0;
    int max_clk;
    std::cin >> max_clk; // 我比较推荐的做法，用来控制总时钟数，防止debug的时候陷入死循环而不自知
    Memory mem;
    mem.load_ins();
    CPU cpu(mem); // 初始化各个元件，读入指令（此处只是示例了memory）
    while(clk < max_clk || finish){ // “finish”表示程序是否运行完毕的状态，大家可以自行设计函数或者变量来表示
        // 各种操作，比如旧值更新成新值
        clk++;
        cpu.step(); // cpu运行一步，包含元件运行一步。当然你也可以不用做“cpu”这一层抽象，可以各个元件独立地写运行
        // 各种操作，旧值更新成新值也可以在此处进行
    }
    std::cout<<"总时钟数: "<<clk<<std::endl; // 输出各种信息
}
```

- ▶ 一个元件只能通过自己当前状态和外部输入来改变状态、进行输出。
- ▶ 请勿使用绝大多数 STL 容器、指针、引用，或动态分配内存空间。
- ▶ 避免全局变量。
- ▶ 模拟器运行耗时短，并不意味着模拟器写得好。可尝试在保证 Tomasulo 架构下减少时钟周期数（不会影响得分）。
- ▶ 建议：各部件存储新、旧两个状态，执行时用自己和其它部件的旧状态运算出新状态，更新时用新状态覆盖（模拟 reg 随时钟更新的时序逻辑）。

- ▶ 想清楚每种指令都怎么处理之后再写，可以画一张设计图，体现各元件的功能、接线与元件间传输的信息。（有「小巧思」拿不准的可以找助教商议）
- ▶ 建议先写一个单级流水的 naïve interpreter!
 - (1) 熟悉 RISC-V 指令功能
 - (2) 方便 debug 对拍：比较每 commit 一条指令后寄存器的状态。
- ▶ 严禁 AI 生成代码或自动补全！请勤于 git commit。

- ▶ 某个周期中，发射了一个需要 x2 的指令，同时 x2 寄存器所依赖的那条指令被 RoB 提交。
- ▶ store 指令的提交过程。
- ▶ 分支预测错误的清空处理。
- ▶ JALR 的处理。
- ▶ Queue / List 接近满时的阻塞。

- ▶ 高级分支预测
- ▶ Cache / 多发射
- ▶ V-Extension / SIMD?



王越天



陈柯杰

仓库结构

- ▶ `issue.pdf`: 作业发布
- ▶ `reference/`: 参考资料
- ▶ `ppt/`: 上课材料
- ▶ `data/`: 包含所有需要通过的数据点

推荐阅读

- ▶ CAAQA 第三章: 指令级并行
- ▶ 计算机体系结构-存储指令的加速 - 知乎
- ▶ 同作者系列文章
- ▶ <https://www.bilibili.com/video/av21376839/>
第五到第九集